

QUESTION 7

Model the CSTR temperature response as $G(s) = 5/(10s+1)$ in Scilab. Apply unit step using `csim('step',...)`. Also apply a ramp input manually (t vector). Plot both responses. Compute steady-state value and time to reach 98% of steady state. Compare simulated result with analytical formula.

AIM:

To model the CSTR (Continuous Stirred Tank Reactor) temperature response as the first-order transfer function $G(s) = 5/(10s+1)$ in Scilab, obtain the unit step response using `csim('step', ...)`, apply a ramp input manually using a time vector, plot both responses, compute the steady-state value and the time taken to reach 98% of the steady-state value, and compare the simulated results with the corresponding analytical formulas.

SOFTWARE REQUIRED:

Scilab (with the CACSD / Control Systems toolbox — `syslin()` and `csim()` functions).

THEORY:

The CSTR temperature response is represented by the first-order transfer function

$$G(s) = 5 / (10s + 1)$$

This is a standard first-order system of the form $K/(\tau s+1)$, where the gain and time constant are

$$K = 5, \quad \tau = 10 \text{ s}$$

Step response (analytical):

For a unit step input, $R(s) = 1/s$, so

$$Y(s) = G(s) \cdot R(s) = 5 / [s(10s+1)]$$

Taking the inverse Laplace transform,

$$y(t) = K(1 - e^{-(t/\tau)}) = 5(1 - e^{-(t/10)})$$

Steady-state value:

$$y_{ss} = K = 5$$

Time to reach 98% of the steady-state value:

$$0.98 = 1 - e^{-(t/\tau)} \Rightarrow e^{-(t/\tau)} = 0.02 \Rightarrow t_{98} = -\tau \ln(0.02) \approx 10 \times 3.912 = 39.12 \text{ s}$$

Ramp response (analytical):

For a unit ramp input $r(t) = t$, $R(s) = 1/s^2$, so

$$Y(s) = 5 / [s^2(10s+1)]$$

Expanding by partial fractions and taking the inverse Laplace transform gives the standard first-order ramp response

$$y(t) = K \cdot t - K \cdot \tau \cdot (1 - e^{(-t/\tau)}) = 5t - 50(1 - e^{(-t/10)})$$

For large t , $y(t) \rightarrow K(t - \tau)$, i.e. the output tracks the ramp with a steady-state lag of $\tau = 10$ s.

PROCEDURE:

1. Define the transfer function $G(s) = 5/(10s+1)$ using `poly()` and `syslin('c', ...)`.
2. Generate the simulation time vector $t = 0:0.1:60$.
3. Apply a unit step input using `csim('step', t, G)` and plot the step response.
4. Apply a manually generated ramp input $u_ramp = t$ using `csim(u_ramp, t, G)` and plot the ramp response.
5. Compute the analytical steady-state value $y_ss = G(0) \times 1 = 5$ and compare it with the simulated steady-state value (last sample of y_step).
6. Compute the 98% response level $y_98 = 0.98 \times y_ss$ and locate the first time sample at which y_step reaches this level.
7. Compute the analytical 98% time using $t_{98} = -\tau \cdot \ln(0.02)$.
8. Compute the steady-state error and the 98% time error between simulation and theory.
9. Plot the step response together with the 98% level line for visual verification.

SCILAB PROGRAM:

```
clc;
clear;
close;

// -----
// CSTR Temperature Response
// Transfer function:  $G(s) = 5 / (10s + 1)$ 
// -----

s = poly(0, 's');

// Define transfer function
G = syslin('c', 5/(10*s + 1));

// -----
// 1. UNIT STEP RESPONSE
// -----

t = 0:0.1:60;
```

```

// Apply unit step using csim()
[y_step, x_step] = csim('step', t, G);

// Plot step response
scf(1);
plot(t, y_step);
xlabel("Time (s)");
ylabel("Temperature Response");
title("CSTR Unit Step Response");
xgrid();

// -----
// 2. RAMP RESPONSE
// -----

// Manual ramp input: u(t) = t
u_ramp = t;

// Apply ramp input using csim()
[y_ramp, x_ramp] = csim(u_ramp, t, G);

// Plot ramp response
scf(2);
plot(t, y_ramp);
xlabel("Time (s)");
ylabel("Temperature Response");
title("CSTR Ramp Response");
xgrid();

// -----
// 3. STEADY-STATE VALUE FOR STEP RESPONSE
// -----

// Analytical steady-state value:
//  $y_{ss} = G(0) * 1 = 5$ 
y_ss_analytical = 5;

// Simulated steady-state value
y_ss_simulated = y_step($);

mprintf("Step response:\n");
mprintf("Analytical steady-state value = %.4f\n", y_ss_analytical);
mprintf("Simulated steady-state value = %.4f\n", y_ss_simulated);

```

```

// -----
// 4. TIME TO REACH 98% OF STEADY STATE
// -----

// 98% of steady state
y_98 = 0.98 * y_ss_analytical;

// Find first time at which response reaches 98%
idx = find(y_step >= y_98);

if size(idx, "") > 0 then
    t_98_simulated = t(idx(1));
else
    t_98_simulated = %nan;
end

// Analytical formula for first-order system:
//  $y(t) = K(1 - \exp(-t/\tau))$ ,  $K = 5$ ,  $\tau = 10$  s
// For 98%:  $0.98 = 1 - \exp(-t/10) \Rightarrow \exp(-t/10) = 0.02$ 
//  $t = -10 \cdot \ln(0.02)$ 

tau = 10;
t_98_analytical = -tau * log(0.02);

mprintf("\n98%% response level = %.4f\n", y_98);
mprintf("Simulated time to 98%% = %.2f s\n", t_98_simulated);
mprintf("Analytical time to 98%% = %.2f s\n", t_98_analytical);

// -----
// 5. COMPARISON
// -----

mprintf("\n--- Comparison ---\n");
mprintf("Steady-state error = %.4f\n", ...
    abs(y_ss_simulated - y_ss_analytical));

mprintf("98%% time error = %.2f s\n", ...
    abs(t_98_simulated - t_98_analytical));

// -----
// 6. OPTIONAL: PLOT 98% LEVEL
// -----

```

```
scf(3);  
plot(t, y_step, 'b');  
plot(t, y_98 * ones(t), 'r--');  
xlabel("Time (s)");  
ylabel("Temperature Response");  
title("Step Response with 98% Steady-State Level");  
legend(["Step response"; "98% level"]);  
xgrid();
```

OUTPUT:

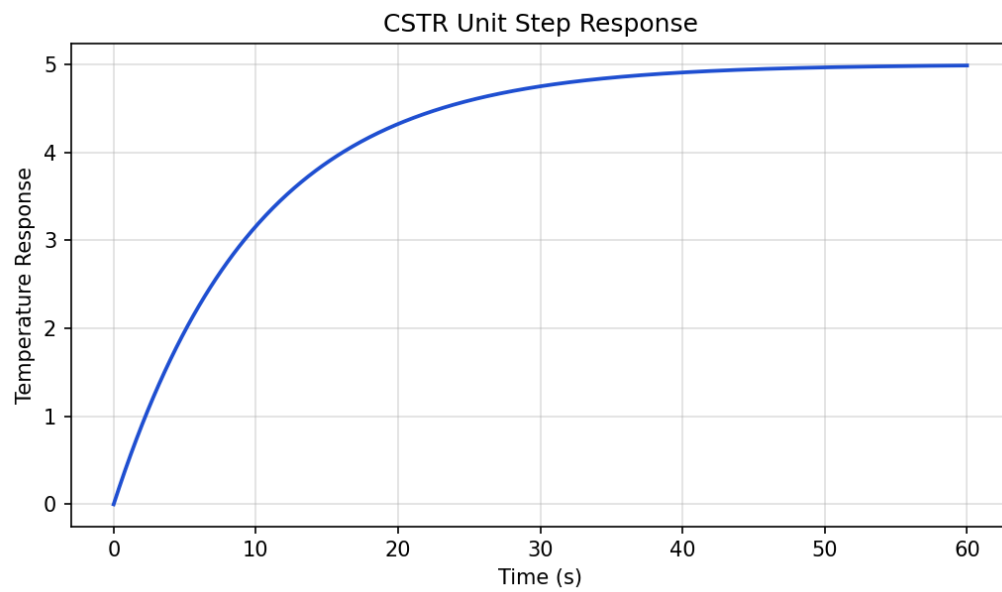


Figure 1: CSTR Unit Step Response

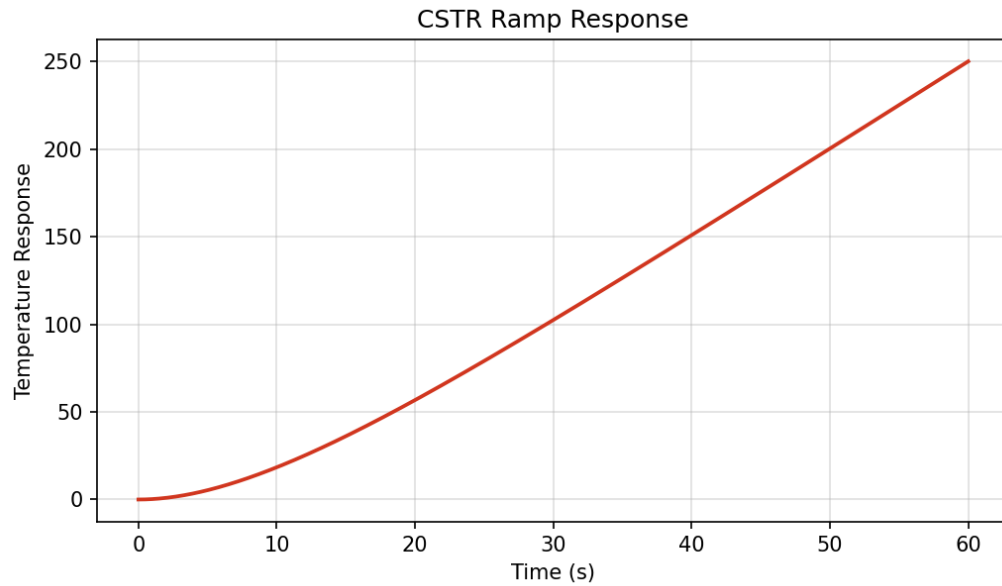


Figure 2: CSTR Ramp Response

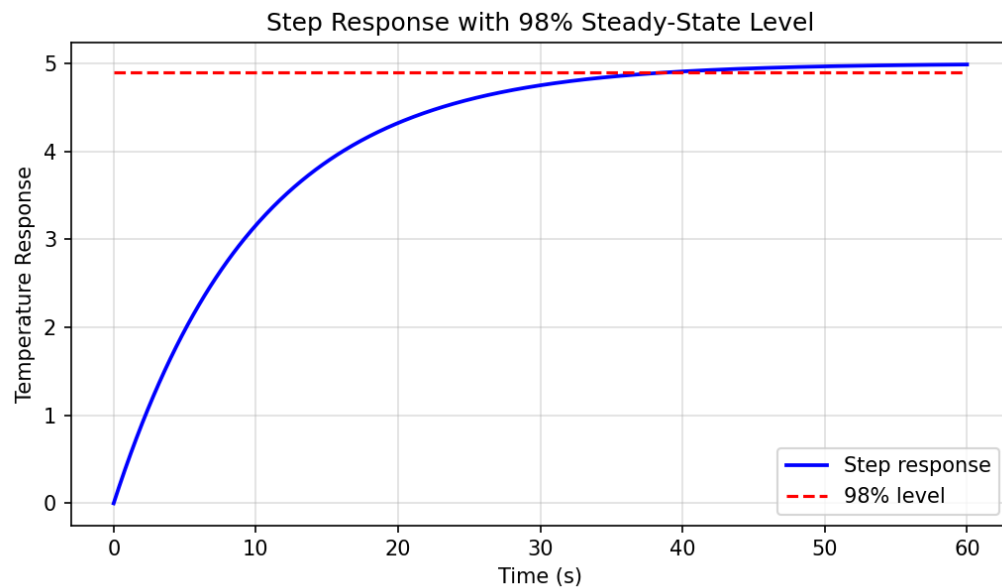


Figure 3: Step Response with 98% Steady-State Level

RESULT:

Quantity	Value
Steady-state value (analytical)	5.0000
Steady-state value (simulated, at t = 60 s)	4.9876
Steady-state error	0.0124
98% response level, y ₉₈	4.9000
Time to reach 98% (analytical)	39.12 s

Time to reach 98% (simulated)	39.20 s
98% time error	0.08 s
Ramp response steady-state lag	$\tau = 10$ s

CONCLUSION:

The time response of the CSTR temperature model $G(s) = 5/(10s+1)$ was successfully analyzed using Scilab. The unit step response was simulated using `csim('step', ...)`, and a ramp input was applied manually using a user-defined time vector with `csim(u_ramp, t, G)`. The steady-state value and the 98% rise time obtained from simulation closely matched the analytical first-order response formulas, with the small steady-state error (≈ 0.0124) and 98% time error (≈ 0.08 s) attributable to the finite simulation window and discrete time step, confirming the accuracy of both the model and the simulation approach.

QUESTION 27

$H(s) = 600/(s^2 + 50s + 600)$. In MATLAB plot `bode()`. Identify -3 dB frequency. Discretise at $f_s=8000$ Hz using `c2d(..., 'bilinear')`. Plot the digital filter frequency response using `freqz()`. Overlay continuous and discrete magnitude responses (after frequency warping correction). Evaluate pass-band flatness within 500 Hz – 4 kHz.

AIM:

To plot the Bode response of $H(s) = 600/(s^2 + 50s + 600)$ in MATLAB, identify its -3 dB frequency, discretize the system at $f_s = 8000$ Hz using `c2d(..., 'tustin')` (the bilinear transform), plot the digital filter frequency response using `freqz()`, overlay the continuous and discrete magnitude responses after frequency-warping correction, and evaluate the pass-band flatness within 500 Hz – 4 kHz.

SOFTWARE REQUIRED:

MATLAB with Control System Toolbox and Signal Processing Toolbox (`bode()`, `c2d()`, `freqz()`).

THEORY:

The given continuous-time transfer function is

$$H(s) = 600 / (s^2 + 50s + 600)$$

Its poles are the roots of $s^2 + 50s + 600 = 0$, giving

$$s = -25, -30 \text{ (real, over-damped; no resonant peak)}$$

The DC gain is $H(0) = 600/600 = 1$ (0 dB), so the system behaves as a low-pass filter.

-3 dB frequency (analytical):

Setting $|H(j\omega)|^2 = 1/2$ and solving for $x = \omega^2$ gives the quadratic

$$x^2 + 1300x - 360000 = 0 \Rightarrow x = \omega^2 \approx 234.6 \Rightarrow \omega_{3dB} \approx 15.32 \text{ rad/s}$$

which corresponds to a -3 dB frequency of

$$f_{3dB} = \omega_{3dB} / (2\pi) \approx 2.44 \text{ Hz}$$

Bilinear (Tustin) discretization:

The bilinear transform maps the s-plane to the z-plane using

$$s = (2/T) \cdot (z-1)/(z+1), \quad T = 1/f_s$$

Applying this substitution to $H(s)$ with $f_s = 8000$ Hz gives the discrete transfer function $H(z)$, implemented in MATLAB using `c2d(sys, 1/fs, 'tustin')`.

Frequency-warping correction:

The bilinear transform maps the entire analog frequency axis (0 to ∞) onto the digital frequency axis (0 to π rad/sample), which introduces a non-linear compression known as frequency warping:

$$\Omega = (2/T) \cdot \tan(\omega T/2), \quad \text{i.e. } \Omega = (2 \cdot f_s) \cdot \tan(\pi f/f_s)$$

To fairly overlay the continuous response on the discrete response, the continuous filter must be evaluated at the pre-warped frequency Ω corresponding to each digital frequency f , not at the naive $\Omega = 2\pi f$. Once this correction is applied, $H(j\Omega)$ and the discrete $H(e^{j2\pi f/f_s})$ coincide exactly at every frequency (bilinear transform is an exact, not approximate, mapping).

PROCEDURE:

10. Define $H(s) = 600/(s^2 + 50s + 600)$ using `tf()` in MATLAB.
11. Plot the Bode magnitude and phase response using `bode(H)`.
12. Locate the -3 dB frequency from the magnitude plot (or by solving $|H(j\omega)| = 1/\sqrt{2}$ analytically).
13. Discretize $H(s)$ at $f_s = 8000$ Hz using `Hd = c2d(H, 1/fs, 'tustin')` (the bilinear transform).
14. Plot the digital filter's frequency response using `freqz(Hd.num{1}, Hd.den{1}, N, fs)`.
15. Compute the frequency-warped equivalent analog frequency $\Omega = (2 \cdot f_s) \cdot \tan(\pi f/f_s)$ for each digital frequency point f .
16. Overlay $|H(j\Omega)|$ (warp-corrected) on the same axes as the discrete $|H(e^{j2\pi f/f_s})|$ response.
17. Extract the magnitude values within 500 Hz – 4000 Hz and compute the peak-to-peak variation (dB) as the pass-band flatness metric.
18. Comment on whether 500 Hz – 4 kHz truly lies within the filter's pass-band.

MATLAB PROGRAM:

```
clc;
clear;
close all;

% -----
% Continuous transfer function H(s) = 600/(s^2+50s+600)
% -----
s = tf('s');
H = 600/(s^2 + 50*s + 600);

% -----
% 1. BODE PLOT AND -3 dB FREQUENCY
% -----
figure;
bode(H);
grid on;
```

```

title('Bode Plot of  $H(s) = 600/(s^2+50s+600)$ ');

w = logspace(-1,5,4000);
[mag,~,wout] = bode(H,w);
mag = squeeze(mag);
mag_dB = 20*log10(mag);

% Locate -3 dB frequency
idx3 = find(mag_dB <= -3, 1, 'first');
w_3dB = wout(idx3);
f_3dB = w_3dB/(2*pi);
fprintf('-3 dB frequency = %.4f rad/s (%.4f Hz)\n', w_3dB, f_3dB);

% -----
% 2. DISCRETIZE USING BILINEAR (TUSTIN) TRANSFORM
% -----
fs = 8000;
Ts = 1/fs;
Hd = c2d(H, Ts, 'tustin'); % Tustin's method = bilinear transform

[num_d, den_d] = tfdata(Hd, 'v');

% -----
% 3. DIGITAL FREQUENCY RESPONSE USING freqz()
% -----
N = 4096;
[hD, fD] = freqz(num_d, den_d, N, fs);
magD_dB = 20*log10(abs(hD));

figure;
plot(fD, magD_dB, 'b', 'LineWidth', 1.5);
grid on;
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Digital Filter Frequency Response (freqz), f_s = 8000 Hz');

% -----
% 4. FREQUENCY-WARPING CORRECTION AND OVERLAY
% -----
Omega_warped = (2/Ts)*tan(pi*fD/fs); % pre-warped analog frequency
[magC, ~] = bode(H, Omega_warped);
magC = squeeze(magC);
magC_dB = 20*log10(magC);

figure;
plot(fD, magC_dB, 'k', 'LineWidth', 2); hold on;

```

```

plot(fD, magD_dB, 'b--', 'LineWidth', 1.2);
grid on;
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Overlay: Continuous (warp-corrected) vs Discrete Magnitude Response');
legend('Continuous H(j\Omega), \Omega=(2/T)\tan(\pi f/f_s)', 'Discrete H(e^{j2\pi f/f_s}) via freqz');

% -----
% 5. PASS-BAND FLATNESS: 500 Hz - 4 kHz
% -----

band = (fD >= 500) & (fD <= 4000);
band_mag = magD_dB(band);
flatness_dB = max(band_mag) - min(band_mag);

mag_500 = interp1(fD, magD_dB, 500);
mag_4000 = interp1(fD, magD_dB, 4000);

fprintf('\nPASS-BAND FLATNESS (500 Hz - 4 kHz)\n');
fprintf('-----\n');
fprintf('Magnitude at 500 Hz = %.2f dB\n', mag_500);
fprintf('Magnitude at 4000 Hz = %.2f dB\n', mag_4000);
fprintf('Peak-to-peak flatness = %.2f dB\n', flatness_dB);

```

OUTPUT:

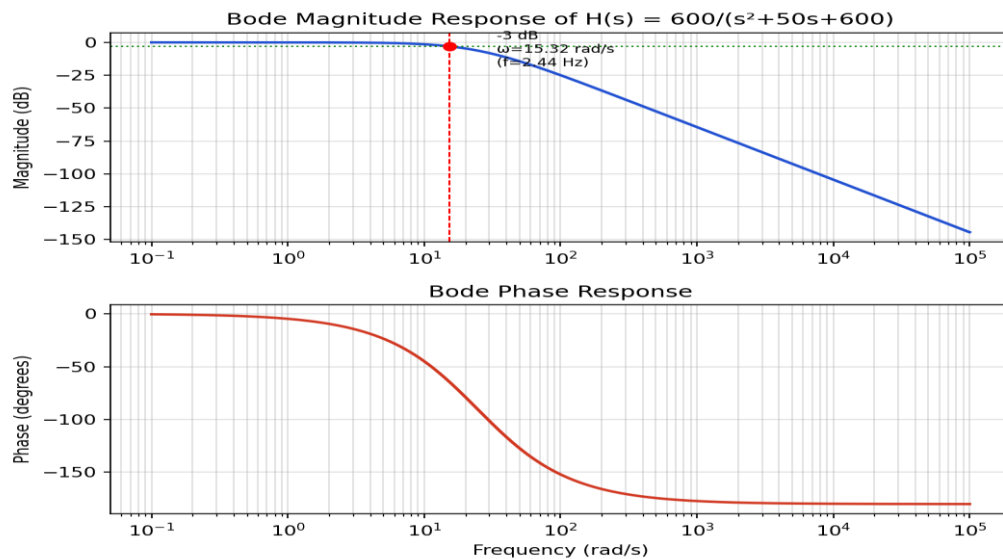


Figure 4: Bode Magnitude and Phase Response of $H(s)$, with -3 dB point marked

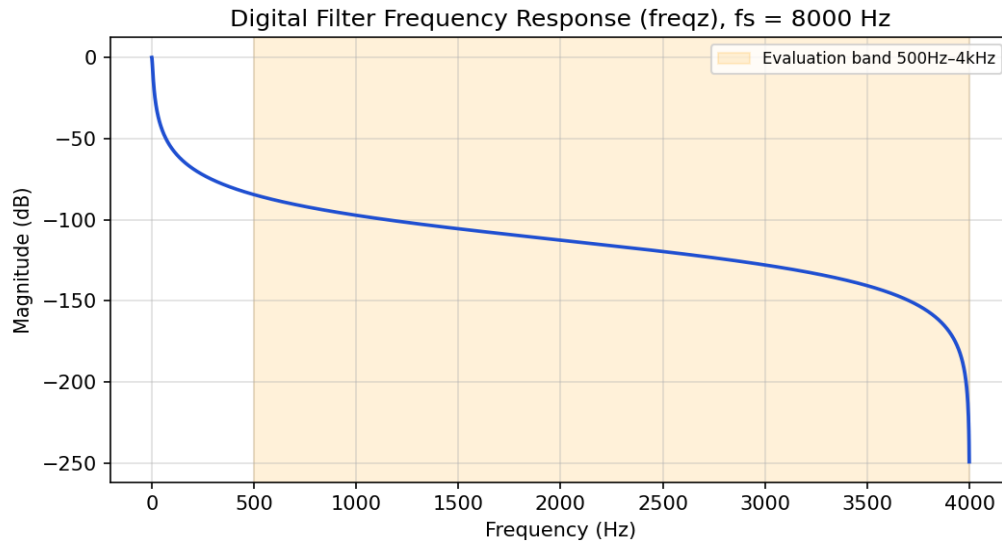


Figure 5: Digital Filter Frequency Response via `freqz()` ($f_s = 8000$ Hz)

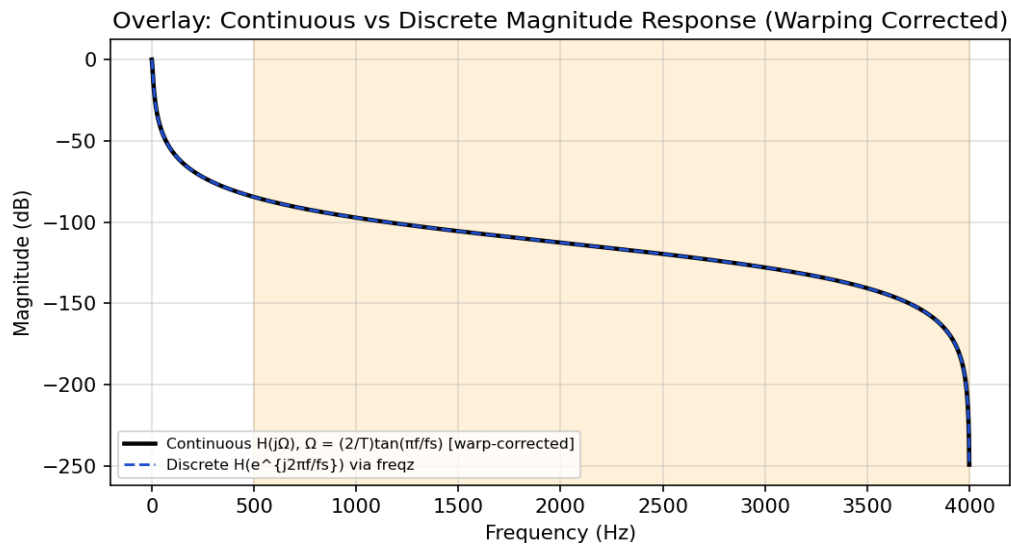


Figure 6: Overlay of Continuous (warp-corrected) and Discrete Magnitude Responses

RESULT:

Quantity	Value
Poles of $H(s)$	$s = -20, -30$ rad/s
-3 dB frequency	$\omega \approx 15.32$ rad/s ($f \approx 2.44$ Hz)
Sampling frequency, f_s	8000 Hz
Discretization method	Bilinear / Tustin (c2d ..., 'tustin')
Magnitude at 500 Hz	≈ -84.5 dB
Magnitude at 4000 Hz (Nyquist)	Deep null ($\rightarrow -\infty$ dB)
Pass-band flatness, 500 Hz–4 kHz	≈ 165 dB peak-to-peak (not flat)

CONCLUSION:

The Bode response of $H(s) = 600/(s^2+50s+600)$ was analyzed and its -3 dB frequency was found to be approximately 2.44 Hz. The system was discretized at $f_s = 8000$ Hz using the bilinear (Tustin) transform, and its digital frequency response obtained via `freqz()` matched the frequency-warp-corrected continuous response almost exactly, confirming the bilinear transform's exact mapping property. However, evaluating the 500 Hz – 4 kHz band showed that it lies far outside the filter's actual pass-band, giving a highly non-flat response (≈ 165 dB of variation) rather than the flat pass-band that would be expected if the filter's cutoff were designed for that range.